OST REPORT

-Vedant Shimpi

-241080068

-SY IT BTECH

Course Outcomes

CO1. Install, configure, and use a variety of open-source tools.
CO2. Utilize open-source IDEs and text editors for software development.
CO3. Collaborate on open-source projects using platforms like GitHub and GitLab.
CO4. Develop and manage projects using version control systems.

Experiment - 1

AIM : Introduction to open source technology, Internet, open source operating systems, open source platform

---

- Introduction to Open Source Technology and its importance.
- Understanding what open source means and how open-source software works.
- Learning about various open-source tools and technologies such as Git, GitHub, Linux, Apache, etc.
- Difference between Open Source and Closed Source software with examples.
- Role of the Internet in open-source development, including collaboration, documentation, hosting, and bug reporting.
- Hands-on introduction to Ubuntu, the open-source operating system used in the lab.
- Learning basic Linux commands like ls, cd, mkdir, sudo, rm -rf, git --version, etc.

- ➔ Understanding why Git is called a Version Control System, including tracking changes, branching, and collaboration.

Conclusion

In this experiment, we gained a clear understanding of what open source technology is and how it promotes transparency, collaboration, and innovation. We explored various open-source tools and technologies such as Linux, Git, GitHub, which play a major role in modern software development. Ubuntu, an open-source operating system, which we used in the lab, providing practical exposure to Linux commands and environment.

Experiment - 2

Aim - Open source development Demographics, sociology and Psychology of open source development, legal issues in open source, Economics of open source, the GNU Project.

1) Write a function reverseArray to reverse the elements of an array in-place.

2) Print an inverted right-angled triangle of stars. n=5

3) Take a number (1–7) and print the corresponding day of the week. Switch case

---

- ➔ Studied the demographics of open source contributors, including age groups, education levels, and global participation.
- ➔ Learned the psychological motivations behind contributing to open source, such as intrinsic interest, recognition, and altruism.

- ➔ Explored legal issues in open source, including licensing, intellectual property rights, compliance, and liability concerns.
- ➔ Examined the economics of open source, including cost reduction, service-based business models, and real-world company success.
- ➔ Studied the GNU Project, its history, its tools (GCC, GDB), and its impact on the Free Software Movement.
- ➔ Understood the basics of GDB (GNU Debugger) and its role in debugging C/C++ programs.

Conclusion -

Open source development is shaped not only by technology but also by the people and communities behind it. Understanding its demographics shows how global and diverse the contributor base is. Legal aspects such as licensing ensure that software remains free, safe, and properly governed. The GNU Project played a foundational role in establishing software freedom and promoting open development.

Experiment - 3

Aim - Version Control Systems Introduction to Git, Basic Git commands (clone, commit, push, pull, branch, merge), Working with GitHub/ GitLab

---

- ➔ Understood what Git is and how it functions as a version control system.
- ➔ Learned how to initialize a repository using git init.
- ➔ Studied user configuration using git config (setting username and email).
- ➔ Practiced basic Git commands such as git status, git add, and git commit.
- ➔ Explored branch-related commands like git branch, git checkout -b, and git merge.

- ➔ Understood how to view commit details using git show and commit history using git log.
- ➔ Performed practical operations shown in screenshots (add, commit, branch creation, merging, viewing history).

Conclusion -

In this experiment, we learned how Git helps manage project versions efficiently and keeps track of all changes. We practiced essential Git commands such as init, add, commit, branch, merge, and log. We also learned how to configure our username and email so Git can properly record our identity in commits. Creating branches and switching between them made it clear how teams work on multiple features simultaneously. The commands git show and git log helped us understand commit history and view detailed changes.

Experiment - 3B

Aim - To understand the basics of GitHub, including how to create a repository, clone, pull, and push files between a local system and GitHub.

- ➔ Learned how to create a new repository on GitHub.
- ➔ Understood how to clone a remote repository to the local system using git clone.
- ➔ Configured Git username and email, and learned how to unset them when needed.
- ➔ Practiced adding, staging, and committing files in Git locally.
- ➔ Learned how to push files from local storage to a remote GitHub repository using git push origin main.
- ➔ Understood how GitHub authentication works through personal access tokens.

- ➔ Learned how to pull updated files from GitHub into the local folder using git pull origin main.
- ➔ Used git status and git log to check changes, commit history, and repository state.

Conclusion -

In this experiment, we learned how GitHub works as a remote hosting service for Git repositories. We practiced pushing files from our local system to GitHub and pulling updates back to the local machine. We also learned how authentication is done using GitHub tokens instead of passwords. Working with push and pull commands helped us understand real-world collaboration workflows. Using git status and git log allowed us to verify file updates and track commit history.

Experiment - 4

Aim - Development Environments Introduction to Integrated Development Environments (IDEs) and Text Editors, Using VSCode, Atom, and Sublime Text, Customizing development environments.

- ➔ Installed different development tools including VSCode, Atom, and Sublime Text on the system.
- ➔ Explored the interface and basic features of each editor/IDE.
- ➔ Checked how to install themes, modify appearance, and change editor settings.
- ➔ Installed important extensions/plugins such as Python extension, linters, and syntax highlighters.

- ➔ Customized the editor environment by changing color themes, font size, and layout preferences.
- ➔ Explored built-in features of VSCode like the debugger, terminal, and extension marketplace.
- ➔ Verified file creation, editing, and saving functions across all three tools.
- ➔ Compared the features, speed, and usability of VSCode, Atom, and Sublime Text.

Conclusion

In this lab, we learned about various text editors and IDEs that developers use for writing and managing code. We installed and explored tools like VSCode, Atom, and Sublime Text to understand their features and differences. We also learned how to customize these editors using themes, extensions, and settings to make the development environment more comfortable and efficient. The practical helped us understand how powerful IDEs and editors are in improving coding productivity.

Experiment - 5

Aim - System Monitoring and Logging Monitoring systems with Nagios/Zabbix, Logging with ELK stack (Elastic search, Log stash, Kibana), Performance analysis and tuning.

- ➔ Studied different system monitoring tools such as Nagios and Zabbix, including how they monitor hosts, services, and network devices.
- ➔ Explored Zabbix features like auto-discovery, triggers, dashboards, and CPU utilization graphs.

- ➔ Observed how Nagios provides host status, process information, and alert histograms for tracking uptime and failures.
- ➔ Learned how the ELK Stack (Elasticsearch, Logstash, Kibana) works together for centralized logging and data visualization.
- ➔ Understood how Logstash collects and processes logs while Elasticsearch stores them for fast searching.
- ➔ Analyzed log data visually using Kibana dashboards to identify patterns, errors, and trends.
- ➔ Compared Nagios, Zabbix, and ELK based on scalability, visualization, alerting, and use cases.
- ➔ Performed basic performance analysis to detect bottlenecks such as CPU load, memory usage, and system errors using these monitoring tools.

Conclusion

In this lab, we learned how system monitoring and logging tools help maintain stable and high-performance systems. Nagios and Zabbix provided insights into server uptime, resource usage, and real-time alerts, while the ELK stack helped us manage and visualize logs effectively. We also understood when each tool is most suitable—Zabbix for scalable modern monitoring, Nagios for basic infrastructure checks, and ELK for advanced log analysis.

Experiment 6

Aim - To contribute to an open source organization. Also to check git logs using Nagios.

- ➔ Studied what Open Source Software is and understood its features such as collaboration, transparency, and community-driven development.
- ➔ Explored how to find suitable open-source repositories on GitHub/GitLab and identified beginner-friendly issues to contribute to.
- ➔ Forked and cloned open-source repositories to a local machine and created new branches to work on improvements.
- ➔ Made contributions such as adding a full project implementation and proposing a new app idea to recognized open-source repositories.
- ➔ Committed and pushed the changes back to GitHub and created pull requests which were reviewed and successfully merged.
- ➔ Learned how Git logs record commit history and how this information can be processed using scripts and visualized using graphs.
- ➔ Understood how Nagios can be integrated with Git using custom plugins to monitor repositories and detect new commits or changes.
- ➔ Visualized commit data (daily, weekly, monthly) using Python for better analysis of contributor activity.

Conclusion -

In this lab, we learned how to effectively contribute to open-source projects by forking repositories, making changes, and submitting pull requests that were successfully merged. This helped us understand real-world collaboration practices used by development teams. We also studied how Git logs maintain version history and how Nagios can be extended with custom scripts to monitor repository activity. By combining Git and Nagios, we can track commit trends, detect new changes, and improve overall project monitoring.